- The biases are right in line with our expectations: if *either* output neuron has a positive weight to a hidden layer neuron, then the bias for the hidden layer neuron is negative. (Neurons 27 and 28 are exceptions and slightly positive.) All rows where both output layer weights are negative have a positive hidden layer bias.

- The positive row filtering for a given neuron sometimes codes exactly as expected, and other times it seems random. However, on closer examination, in every single case where it seems random, the weight from the other output neuron to that row is significantly larger. It is really coding for the other neuron in those rows, and simply looks random because we are sorting by the wrong set of clauses.

- The output neuron weights partition the hidden layer neurons into three categories: (1) positive and larger weight from output neuron 0, (2) positive and larger weight from output neuron 1, and (3) negative from both neurons. Neurons of type (1) code for output neuron 0 and function f_0. Neurons of type (2) code for neuron 1. Neurons of type (3) prove negative witnesses. This is visualized clearly in the bottom right panel of Figure 23. There, each row is sorted according to which cell of the partition is in: for rows of type (1), we sort by the clauses of f_0, for rows of type (2), we sort by the clauses of f_1, and rows of type (3) are filtered out. We see that the result is nearly perfect coding.

- Once the system has chosen which output a row is coding for, it effectively ignores the other output for that row - the other weight has a much smaller magnitude weight, and that weight has a tendancy to be (slightly) positive.

- In Section 4, we saw that even as we increased the number of clauses, the network was not able to learn a weight distribution with more than roughly half positive rows. We here see a fairly even divide between the three cells of the partition, and so the number of negative rows is only 12. This is additional evidence that the system does not need as many negative rows as the solution that it is converging to has. For this case where there are 2 output neurons, one could also argue that it is seeking equality between the positive rows and the negative rows, and the negative rows are reused between the two output neurons. However, (a) we do not see that same effect with 3 output neurons (below), and (b) even if that were the case, it does imply a limit on how much logic can be built into the negative rows (since they are the same between the two outputs, with relatively minor differences in how negative the output layer weight is).

We next turn to the case where we have 3 output neurons. The setup is the same otherwise, except that we now train with more data (200,000 inputs). The result was slightly worse performance for this more difficult case (since we now have 24 total clauses, instead of 16). Specifically, after 1000 epochs, we have training loss 5.86% and test set accuracy (all 3 outputs correct) of 95.05%. Figure 24 depicts the results for the following set of formulas:

$$\begin{aligned} g_0(x) = (x_5 \wedge x_{10} \wedge x_{11} \wedge x_{20}) \vee (x_4 \wedge x_7 \wedge x_{14} \wedge x_{27}) \vee (x_{12} \wedge x_{17} \wedge x_{21} \wedge x_{24}) \vee \\ (x_{13} \wedge x_{15} \wedge x_{28} \wedge x_{30}) \vee (x_2 \wedge x_6 \wedge x_9 \wedge x_{22}) \vee (x_8 \wedge x_{23} \wedge x_{29} \wedge x_{31}) \vee \\ (x_0 \wedge x_{16} \wedge x_{25} \wedge x_{26}) \vee (x_1 \wedge x_3 \wedge x_{18} \wedge x_{19}) \end{aligned}$$

$$g_1(x) = (x_0 \wedge x_{10} \wedge x_{19} \wedge x_{27}) \vee (x_{11} \wedge x_{12} \wedge x_{14} \wedge x_{24}) \vee (x_2 \wedge x_3 \wedge x_4 \wedge x_7) \vee (x_6 \wedge x_8 \wedge x_9 \wedge x_{20}) \vee (x_{16} \wedge x_{22} \wedge x_{26} \wedge x_{30}) \vee (x_1 \wedge x_{21} \wedge x_{25} \wedge x_{29}) \vee (x_{17} \wedge x_{23} \wedge x_{28} \wedge x_{31}) \vee (x_5 \wedge x_{13} \wedge x_{15} \wedge x_{18})$$

$$g_2(x) = (x_1 \wedge x_5 \wedge x_{21} \wedge x_{30}) \vee (x_6 \wedge x_{12} \wedge x_{13} \wedge x_{20}) \vee (x_2 \wedge x_{17} \wedge x_{22} \wedge x_{29}) \vee (x_{10} \wedge x_{26} \wedge x_{28} \wedge x_{31}) \vee (x_4 \wedge x_{11} \wedge x_{15} \wedge x_{25}) \vee (x_0 \wedge x_7 \wedge x_{16} \wedge x_{18}) \vee (x_3 \wedge x_8 \wedge x_{23} \wedge x_{27}) \vee (x_9 \wedge x_{14} \wedge x_{19} \wedge x_{24})$$

This solution is very similar to the case with 2 output neurons, and as we can see in Figure 24, it is clearly using feature channel coding. However the structure of the network is not quite as clean. This is not surprising, given (a) the poorer performance, and (b) the fact that we are starting to approach the limit we discussed in Section 4, where the total number of clauses equals the size of the hidden dimension. Specifically, we see that the layer 1 biases are not as consistently negative for the positive rows, and that not every positive row chose a clear output to code for (see neuron 27, which just contributes a bit of positive value from all input variables. It is however, interesting to note that the number of negative rows is only 3 (or 4 if you count row 19), leaving each output about 9 rows of positive coding, again providing evidence that the single output solutions are over investing in negative rows.

We did also experiment with 4 output neurons, but our initial attempts at that did not yield particularly effective networks. In that case the number of clauses was equal to the size of the hidden layer; the trained networks tended to have error rates around 1/3. Still even in these poorly performing networks, it was clearly trying to use feature channel coding.

8 Further Research

We believe the combinatorial approach and its first application through feature channel coding, can have implications even before we have the ability to apply it to large real world networks. In particular, in the same way we used it to understand scaling laws, one can attempt to understand other important neural computation aspects such as sparsity, quantization, and the linear representation hypothesis, to name a few. Understanding to what extent computation in real production neural networks contains Boolean features and tehir codes is also of great interest. Finally, it has not escaped our notice that the specific coding we have postulated immediately suggests a possible computation mechanism for neural tissue.

References

[1] Micah Adler and Nir Shavit. On the complexity of neural computation in superposition. *arXiv preprint arXiv:2409.15318*, 2024.

[2] Emmanuel Ameisen et al. Circuit tracing: Revealing computational graphs in language models. `https://transformer-circuits.pub/2025/attribution-graphs/methods.html`, 2025.

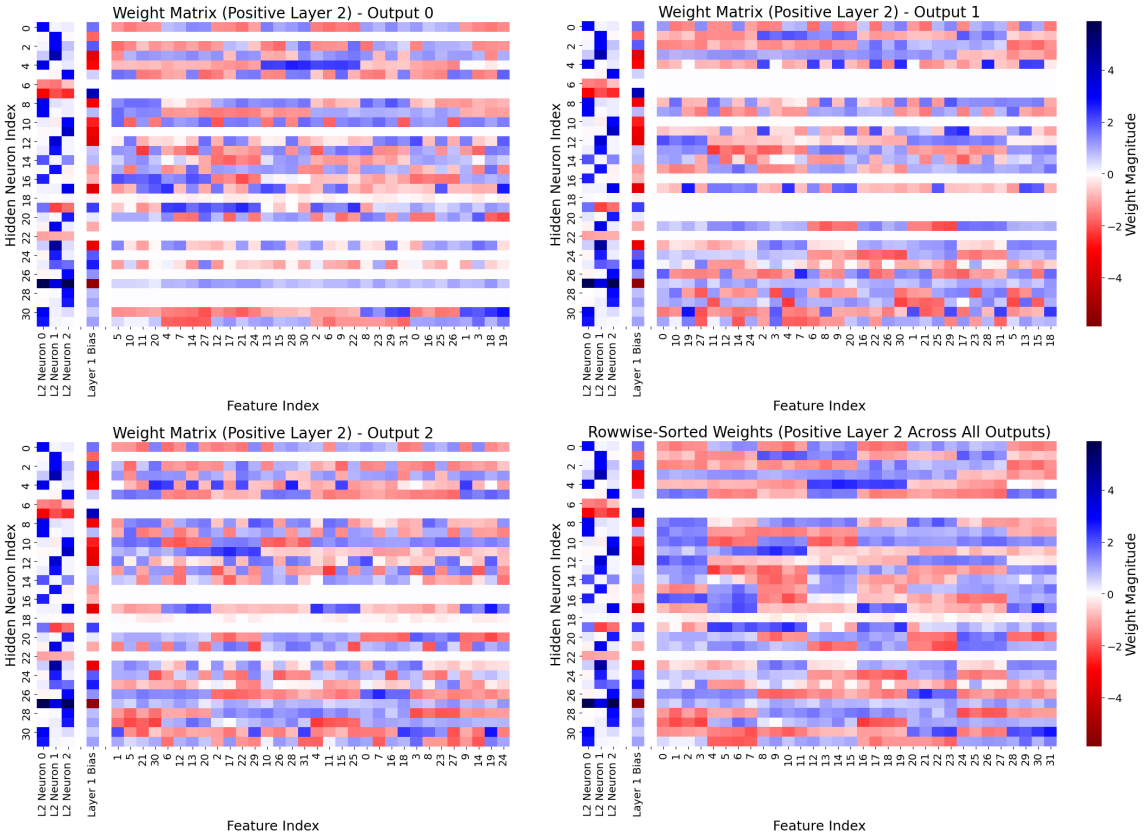

Figure 24: Trained neural network for g_0, g_1, and g_2. The top right is sorted by the clauses of g_0 (computed by output 0), and filtered to show only those neurons that have a positive edge to output neuron 0. The top left is the equivalent version for output neuron 1 and g_1, and the bottom left is the equivalent for output neuron 2 and g_2. The bottom right has all rows with a positive edge to any output neuron, and each such row is sorted by the clauses corresponding to the output neuron with the largest weight to that row/hidden layer neuron.

[3] Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits*, 2025.

[4] Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence*, 35(8):1798–1828, 2013.

[5] Thomas Bricken et al. Towards monosemanticity: Decomposing language models with dictio-